

BCA182 EMBEDDED SYSTEMS PROGRAMMING

LABORATORY ACTIVITY NO. 1

FreeRTOS-Based STM32

Prepared by: Dja-ver Q. Hassan BSCA – 4th Year

Course: BCA182 Embedded Systems Programming

Prepared for: Asst. Prof. Paul Rodolf P. Castor, M.Sc.

Date: September 28, 2026

Report status

Core design, architecture, implementation, and static-analysis sections are complete. Formal FT-01 to FT-10 observations and the three deliberate FreeRTOS fault-experiment observations are intentionally left as pending until they are actually performed and recorded.

Report Overview

RoomSense Monitor is a simulated STM32F103C8T6 room-monitoring system implemented with PlatformIO, the STM32Cube framework, STM32 HAL, native FreeRTOS APIs, and Wokwi. The design separates sensing, display control, user input, motion monitoring, alarm evaluation, and operating-state control into concurrent tasks rather than a single polling loop.

The implementation uses FreeRTOS queues for data transfer, an event group for shared state/event signaling, a mutex for serial-output protection, blocking delays for cooperative CPU usage, and `vTaskDelayUntil()` for periodic sensing. The system monitors temperature, humidity, ambient light, and motion, provides rotary-encoder navigation on an OLED, activates a buzzer outside the 18 C to 30 C temperature range, and transitions between ACTIVE and INACTIVE states according to motion activity.

Required Report Sections

Section	Purpose
1	Problem and Requirements
2	System Architecture and Design
3	FreeRTOS Architecture
4	Implementation
5	Verification and Testing
6	Static Code Analysis
7	Engineering Discussion
8	Conclusion

Evidence policy

The laboratory handout states that a functional test must not be marked PASS without recording the actual observed behavior. Therefore, this report does not fabricate FT-01 to FT-10 outcomes or fault-experiment observations.

1. Problem and Requirements

The laboratory problem is to build a FreeRTOS-based STM32 room multisensor that can monitor environmental conditions and respond to user input in a simulated embedded environment. The project must use the STM32 Blue Pill target, PlatformIO with the STM32Cube framework, STM32 HAL drivers, and native FreeRTOS mechanisms. Arduino framework abstractions are not used.

1.1 Functional Requirements

ID	Requirement	Implemented behavior
FR-01	Temperature measurement	Periodically obtain temperature data.
FR-02	Humidity measurement	Periodically obtain humidity data.
FR-03	Ambient-light measurement	Monitor relative ambient-light level.
FR-04	Motion detection	Detect simulated motion using the PIR sensor.
FR-05	OLED display	Display one selected measurement at a time.
FR-06	Rotary encoder navigation	Switch among Temperature, Humidity, Light, and Motion.
FR-07	Temperature alarm	Activate the buzzer outside the configured normal range.

ID	Requirement	Implemented behavior
FR-08	Activity state	Support ACTIVE and INACTIVE states.
FR-09	Automatic inactivity	Enter INACTIVE after the inactivity duration expires.
FR-10	Automatic reactivation	Return to ACTIVE when motion is detected.

1.2 Wokwi Adaptation

The project is designed for Wokwi because physical laboratory hardware was not available. Wokwi provides controllable virtual peripherals that make functional behavior repeatable, but it does not reproduce all electrical characteristics of real hardware. The project therefore treats the simulator as a functional-development environment rather than a complete substitute for physical validation.

A simulation-specific FreeRTOS timing/port adaptation is used so the scheduler behaves reliably in the Wokwi STM32F103 environment. This adaptation is an implementation accommodation for simulation and should be reviewed or replaced by the standard Cortex-M FreeRTOS port when moving to physical STM32 hardware.

2. System Architecture and Design

2.1 Hardware Architecture

RoomSense Monitor - System Architecture

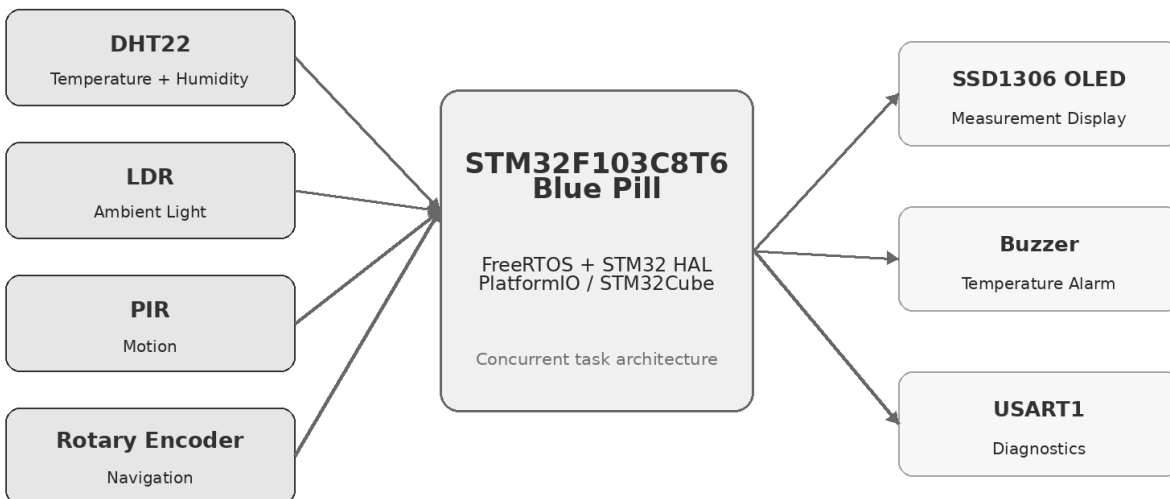


Figure 1. Hardware and top-level software architecture of the RoomSense Monitor.

Component	Purpose	STM32 Pin / Interface
DHT22	Temperature and humidity	PA1
LDR	Ambient light	PA0 / ADC
PIR sensor	Motion detection	PA3
Rotary encoder CLK	Navigation input	PA4
Rotary encoder DT	Navigation input	PA5

Component	Purpose	STM32 Pin / Interface
SSD1306 OLED	Information display	PB6/PB7 (I2C)
Buzzer	Temperature alarm	PA2
USART1	Diagnostic output	PA9/PA10

2.2 Software Decomposition

The software is decomposed by responsibility. Sensor acquisition is isolated from display rendering and alarm evaluation. Input processing is isolated from display ownership. Motion detection and system-state management are separated so that the state machine can make policy decisions without directly owning every peripheral. This decomposition reduces hidden coupling and makes hardware-independent decision logic easier to test.

2.3 State Machine

RoomSense Monitor - ACTIVE / INACTIVE State Machine

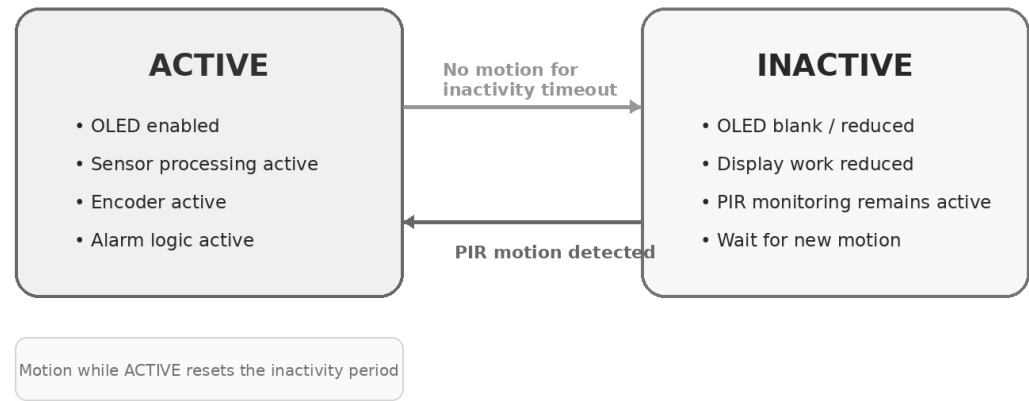


Figure 2. ACTIVE/INACTIVE state transitions used by the application.

In ACTIVE, the OLED, normal sensor processing, encoder navigation, and alarm logic are available. After the configured inactivity interval, the system enters INACTIVE, blanks or reduces display activity, and continues PIR monitoring. A later PIR event returns the system to ACTIVE.

2.4 State Transition Table

Current State	Condition	Next State	System response
ACTIVE	Motion detected	ACTIVE	Refresh the activity period and continue normal operation.
ACTIVE	Inactivity timeout reached	INACTIVE	Blank/reduce OLED activity while preserving motion monitoring.
INACTIVE	No motion	INACTIVE	Remain in reduced-activity state.
INACTIVE	PIR motion detected	ACTIVE	Restore normal display and interaction behavior.

Design implication

Motion monitoring remains operational in both states. This prevents the power-saving / reduced-display state from becoming a dead end and gives the state machine a clear reactivation path.

3. FreeRTOS Architecture

RoomSense Monitor - FreeRTOS Task Communication

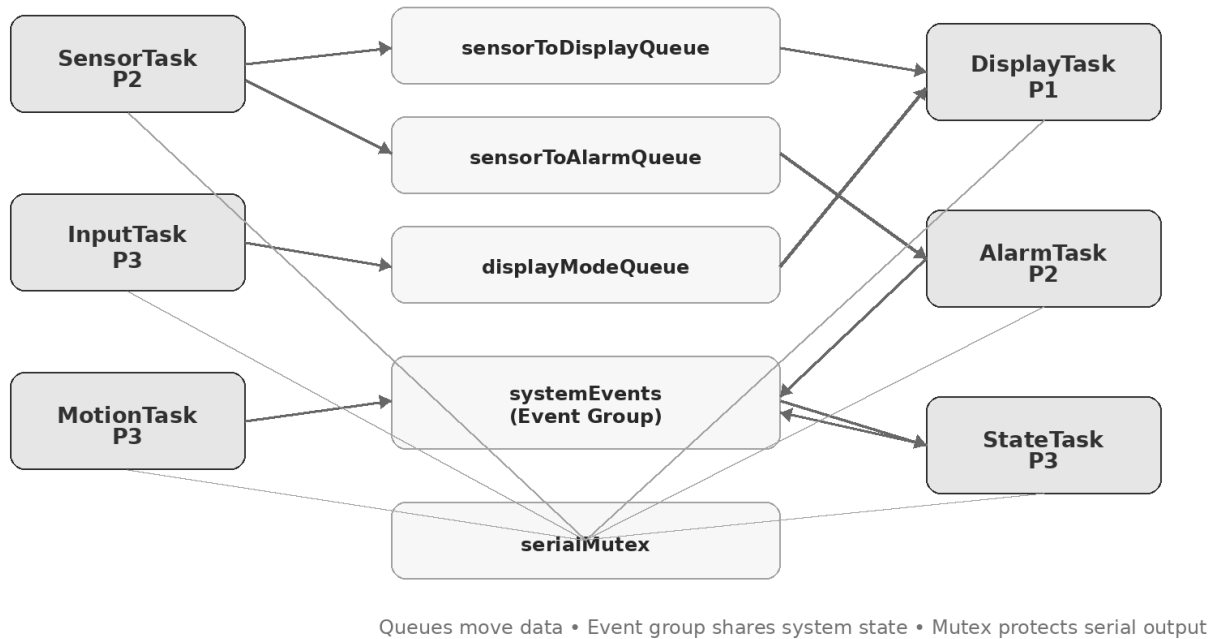


Figure 3. FreeRTOS task communication and shared synchronization objects.

3.1 Required Task Table

Task	Responsibility	Trigger / Period	Priority	IPC	Typical Blocked Condition
SensorTask	Read DHT22 and LDR	Periodic (2 s design target)	2	Queues	Delay / vTaskDelayUntil
DisplayTask	Own and update OLED	Event / data update	1	Queues + event group	Waiting for data/update
InputTask	Read rotary encoder	Short periodic / input event	3	Queue	Delay / wait
MotionTask	Monitor PIR	Periodic / motion event	3	Event group	Delay / wait
AlarmTask	Evaluate temperature and buzzer	Sensor update	2	Queue + event group	Waiting for data
StateTask	Manage ACTIVE/INACTIVE	Motion / timeout	3	Event group	Delay / wait

3.2 Priority Rationale

InputTask, MotionTask, and StateTask use priority 3 because user interaction, motion response, and system-state changes should be responsive. SensorTask and AlarmTask use priority 2 because they perform important but less latency-sensitive periodic/data-driven work. DisplayTask uses priority 1 because OLED refresh can tolerate more latency than control or input paths. The intent is responsiveness without allowing a continuously ready high-priority task to monopolize the processor.

3.3 Inter-Task Communication and Synchronization

- `sensorToDisplayQueue` transfers the latest sensor data from SensorTask to DisplayTask.
- `sensorToAlarmQueue` transfers sensor information to AlarmTask without coupling alarm logic to the sensor driver.
- `displayModeQueue` carries rotary-encoder navigation changes from InputTask to DisplayTask.
- `systemEvents` shares system conditions such as ACTIVE, motion, alarm, and PIR level.
- `serialMutex` protects shared serial diagnostics so task messages do not overlap.
- Blocking waits and delays keep tasks out of the Ready state when they have no immediate work.

3.4 Periodic Timing

SensorTask uses `vTaskDelayUntil()` so periodic sampling is anchored to the previous scheduled wake time. This reduces accumulated drift compared with delaying for a fixed interval after the task finishes each iteration. The project also uses a simulation-specific timer arrangement for stable RTOS/HAL timing under Wokwi.

4. Implementation

4.1 Sensor Acquisition

SensorTask owns periodic environmental acquisition. The DHT22 provides temperature and humidity, while the LDR is sampled through the ADC for a relative light reading. The task packages the latest values and distributes them to consumers through queues instead of exposing mutable shared data directly.

4.2 Display and Navigation

DisplayTask owns the SSD1306 OLED. This ownership rule avoids concurrent writes to the display driver. InputTask reads the rotary encoder and publishes the selected display mode through `displayModeQueue`. The user can rotate through Temperature, Humidity, Light, and Motion pages.

4.3 Alarm Logic

AlarmTask receives sensor data and evaluates temperature against the laboratory thresholds. The normal range is bounded by 18 C and 30 C. Values below the lower threshold or above the upper threshold request the buzzer alarm; values within the permitted range clear the alarm condition. Keeping this decision logic independent of the buzzer driver makes the boundary behavior suitable for unit testing.

4.4 Motion and Activity State

MotionTask monitors the PIR input and publishes activity information through the event group. StateTask centralizes the ACTIVE/INACTIVE decision. When motion is absent long enough to reach the configured inactivity timeout, the system enters INACTIVE. Motion while INACTIVE causes reactivation.

4.5 Serial Diagnostics

Multiple tasks may emit diagnostic messages. A FreeRTOS mutex serializes access to the shared UART output. This is a practical shared-resource protection example: without a mutex, multi-part messages can interleave when task execution switches between writes.

4.6 Wokwi-Specific Scheduler Consideration

During simulation, the project required a Wokwi-specific FreeRTOS port/tick arrangement to avoid scheduler startup and context-switch issues in the simulated STM32F103 environment. The workaround keeps the application architecture intact while adapting low-level scheduling behavior for the simulator. On physical hardware, the standard FreeRTOS Cortex-M port and normal STM32 timebase configuration should be preferred and revalidated.

5. Verification and Testing

5.1 Unit-Test Coverage

The laboratory requires a minimum of 13 meaningful unit tests: five for alarm boundary logic, four for navigation behavior, and four for the ACTIVE/INACTIVE state machine. The project test suite is structured around those decision points.

Category	Required cases	Coverage status
Temperature alarm	Below 18 C; exactly 18 C; normal; exactly 30 C; above 30 C	Implemented in test suite
Display navigation	Forward; reverse; forward wraparound; reverse wraparound	Implemented in test suite
System state	ACTIVE/no timeout; ACTIVE/timeout; INACTIVE/no motion; INACTIVE/motion	Implemented in test suite

Final execution evidence

Before submission, run `pio test` and replace this note with the final pass/fail summary from the actual command output. The laboratory requires all final tests to pass.

5.2 Wokwi Functional Verification

ID	Stimulus	Expected Result	Actual Observation	Result
FT-01	Change temperature	Displayed temperature updates.	To be recorded	PENDING
FT-02	Change humidity	Displayed humidity updates.	To be recorded	PENDING
FT-03	Change light input	Light value changes.	To be recorded	PENDING
FT-04	Rotate encoder clockwise	Next page is selected.	To be recorded	PENDING
FT-05	Rotate encoder counterclockwise	Previous page is selected.	To be recorded	PENDING
FT-06	Set temperature above 30 C	Alarm activates.	To be recorded	PENDING
FT-07	Return temperature to normal	Alarm stops.	To be recorded	PENDING
FT-08	Trigger PIR	System is ACTIVE.	To be recorded	PENDING
FT-09	Allow inactivity timeout	System becomes INACTIVE.	To be recorded	PENDING
FT-10	Trigger PIR while INACTIVE	System returns to ACTIVE.	To be recorded	PENDING

The table is intentionally left pending until each Wokwi stimulus is actually applied and its observed behavior is recorded. This follows the laboratory rule that a test cannot be marked PASS without an actual observation.

5.3 Deliberate FreeRTOS Fault Experiments

Experiment	Temporary fault	What to observe	Recorded result
1. Remove blocking	Remove delay/blocking from a continuously executing task	Scheduling, responsiveness, CPU usage, starvation risk	Pending actual experiment
2. Change priority	Assign an unnecessarily high priority to a frequent task	Responsiveness of other tasks; scheduling imbalance	Pending actual experiment
3. Remove mutex	Temporarily remove serial mutex protection	Interleaved or corrupted serial diagnostics	Pending actual experiment

Each experiment must be temporary. After observation, the correct delay, intended priority, and mutex protection must be restored before the final build.

6. Static Code Analysis

PlatformIO static analysis was executed with `pio check`. The run completed with PASSED status and reported 0 high-severity findings, 0 medium-severity findings, and 6 low-severity style findings. The six findings were pointer-constness recommendations in FreeRTOS/HAL callback-style function signatures.

File / line	Finding summary	Severity	Interpretation / action
src/rtos_hooks.c:20	pcTaskName pointer could be const	Low / style	Retained to match FreeRTOS hook signature expectations.
src/rtos_hooks.c:32	htim pointer could be const	Low / style	Reviewed; callback/signature compatibility preserved.
src/stm32f1xx_hal_msp.c:89	hi2c pointer could be const	Low / style	HAL-generated/API signature retained.
src/stm32f1xx_hal_msp.c:124	hi2c pointer could be const	Low / style	HAL-generated/API signature retained.
src/stm32f1xx_hal_msp.c:155	huart pointer could be const	Low / style	HAL-generated/API signature retained.
src/stm32f1xx_hal_msp.c:195	huart pointer could be const	Low / style	HAL-generated/API signature retained.

No high- or medium-severity issue required a functional code change. The low-severity findings were reviewed rather than blindly modified because changing callback signatures can create incompatibility with framework or HAL APIs. This is consistent with the laboratory requirement to interpret static-analysis findings instead of only presenting tool output.

7. Engineering Discussion

7.1 Architectural Trade-offs

Separating the application into multiple tasks increases clarity and responsiveness but also introduces synchronization requirements. Queues and event groups add structure and safety, although they consume RAM and require careful ownership rules. A single-loop design would be simpler, but it would not demonstrate the required real-time concurrency concepts and would make blocking peripheral work more likely to delay unrelated functions.

7.2 Priority and Blocking Trade-offs

High priorities are reserved for work that should respond quickly, but priority alone does not guarantee a good real-time design. A high-priority task that never blocks can prevent lower-priority tasks from running often enough. The design therefore combines explicit priorities with blocking delays and event/data waits. The deliberate fault experiments are intended to demonstrate this behavior directly.

7.3 Simulation Limitations

- Wokwi does not reproduce real electrical noise, supply variation, wiring resistance, or all peripheral timing behavior.
- DHT22 and PIR behavior in simulation is deterministic compared with physical sensors.
- The LDR value represents a simulated analog level rather than a calibrated lux measurement.
- The scheduler/tick workaround is specific to the simulated STM32 environment and should not be assumed necessary on physical hardware.
- Physical validation is still required before claiming hardware reliability.

7.4 Debugging and Problems Encountered

The most significant integration issue was obtaining stable FreeRTOS scheduling behavior under the simulated STM32F103 environment. The resolution was to use a simulation-compatible low-level FreeRTOS/timer arrangement while leaving application-level tasks, queues, event groups, mutexes, and state logic unchanged. This preserved the educational architecture while acknowledging the limitations of the simulator.

7.5 Alternative Approaches

A hardware build could use the standard STM32 FreeRTOS Cortex-M port, validate timing with a logic analyzer, and calibrate the LDR against known illumination levels. Notifications could replace some queue/event-group paths when only a single consumer needs a lightweight wake-up signal. The current design favors clarity and traceability over maximum memory optimization.

8. Conclusion

RoomSense Monitor demonstrates how a small embedded application can be organized as a concurrent FreeRTOS system rather than a monolithic loop. The project integrates sensor acquisition, OLED output, user navigation, motion monitoring, alarm behavior, and an activity state machine while using queues, an event group, a mutex, priorities, blocking, and periodic scheduling for well-defined roles.

The work also shows that simulation is valuable for functional development but introduces its own constraints. The Wokwi-specific scheduler adaptation solved a simulator integration problem, while the application design remained transferable to physical STM32 hardware. Static analysis found no high- or medium-severity problems, and the remaining low-severity items were reviewed in the context of framework callback signatures.

The next improvements are to complete the formal FT-01 to FT-10 observation table, perform and document the three deliberate FreeRTOS fault experiments, run the final unit-test suite, and then validate the same architecture on physical STM32 hardware. Additional future work could include configurable thresholds, data logging, more environmental sensors, and richer OLED pages.

Appendix A - Requirements Traceability Matrix

Requirement	Implementation	Verification evidence
FR-01	SensorTask / DHT22	Unit sensor support + FT-01
FR-02	SensorTask / DHT22	FT-02

Requirement	Implementation	Verification evidence
FR-03	SensorTask / LDR ADC	FT-03
FR-04	MotionTask / PIR	FT-08
FR-05	DisplayTask / OLED	FT-01 to FT-03
FR-06	InputTask + displayModeQueue	Navigation unit tests + FT-04/FT-05
FR-07	AlarmTask + buzzer	Alarm unit tests + FT-06/FT-07
FR-08	StateTask + systemEvents	State unit tests + FT-08 to FT-10
FR-09	MotionTask / StateTask	State unit tests + FT-09
FR-10	MotionTask / StateTask	State unit tests + FT-10

The matrix links every major functional requirement to its implementation location and the verification evidence expected by the laboratory. Final PASS status should be assigned only after the corresponding tests are executed and actual observations are recorded.

Appendix B - Final Submission Verification

Item	Command / Evidence	Status
Firmware build	pio run	Run before final submission
Unit tests	pio test	Run before final submission
Static analysis	pio check	PASSED: 0 high, 0 medium, 6 low
Functional verification	FT-01 to FT-10 observation record	Pending final recorded observations
Fault experiments	Remove blocking / priority / mutex experiments	Pending actual observations
Repository status	git status	Check clean working tree
Remote repository	git push	Push final source and documentation

References

1. BCA182 Embedded Systems Programming - Laboratory Activity 1 handout, September 2026.
2. FreeRTOS application APIs and kernel concepts used by the project: tasks, queues, mutexes, event groups, and vTaskDelayUntil().
3. STM32Cube / STM32 HAL interfaces used for GPIO, ADC, I2C, UART, and timer configuration.
4. Wokwi simulator used as the hardware simulation environment for the STM32 Blue Pill and connected peripherals.